Feyman AHRS Device Protocol Reference

For Feyman AHRS Tool 0.1.8 · 2026-10-15

This reference follows the current desktop implementation and the confirmed device behavior. It covers serial measurements, magnetometer calibration, system calibration, and AT commands used by the tool. The original vendor guide remains in the project records, but some of its mode values, calibration commands, frame lengths, thresholds, and status bits differ from this firmware. Validate any new model or firmware on hardware.

1. Serial and command rules

The application uses fixed settings: 115200 bit/s, 8 data bits, no parity, 1 stop bit, and no flow control. AT commands contain printable ASCII and end in CRLF (bytes `0D 0A`). Text lines and binary frames can be interleaved on the same serial stream. A single serial read is not necessarily a complete message.

Command	Current purpose and completion condition
`AT+THIS_PORT`	Query the device UART; wait for both `OK` and `This Port is UartN`.
`AT+WORK_MODE=2`	Set AHRS mode during connection initialization; continue after `OK`.
`AT+AHRS_OUTPUT=UARTN,ON`	Enable AHRS output on the identified device UART. `N` is not the computer COM number.
`AT+AHRS_OUTPUT=UARTN,OFF`	Disable AHRS output on the specified device UART without closing the computer port.
`AT+VERSION`	Read a nonempty `Version:` reply.
`AT+READ_PARA=ALL`	Read parameters and retain the original device text.
`AT+MAG_CALI_CFG=2D` / `3D`	Choose 2D (4-parameter) or 3D (8-parameter) magnetic calibration.
`AT+MAG_CALI_MANU=START`	Start a magnetometer calibration run.
`AT+MAG_CALI_MANU=STOP`	Stop the run without saving its result.
`AT+MAG_CALI_MANU=SAVE`	Attempt to save the result after the UI indicator checks pass.
`AT+CALIBRATE_MODE=STEP1`	Begin the bottom-face and side-face static system calibration step.
`AT+CALIBRATE_MODE=STEP2`	Begin the subsequent six-face static step.
`AT+SYSTEM_RESET`	Reboot; allow about 3 seconds for changed parameters to auto-save first.

Manual commands queue for transmission without waiting for a reply or interrupting an active automatic command. Automatic commands check the corresponding reply. A system calibration step is confirmed by a matching `$GPCAL` line, not by an arbitrary `OK`. Firmware upgrade uses its own flow; do not send upgrade commands from the manual input.

2. Common binary frame

fmiA and fmib use little-endian multibyte values. Offsets start at zero. The length field counts payload bytes only. CRC16/XMODEM covers only the payload and is stored least-significant byte first.

Region	Size	Content
Header	4 bytes	Case-sensitive ASCII message ID.
Payload length	2 bytes	Little-endian `uint16`.
Payload	As specified	Time and measurement values.
CRC	2 bytes	Little-endian CRC16/XMODEM.

```
frame_bytes = payload_bytes + 8
crc_initial = 0x0000
crc_polynomial = 0x1021
crc_test("123456789") = 0x31C3
```

Do not include the header, length, or CRC field in the CRC input. A receiver must buffer partial frames, handle consecutive frames, and resynchronize after an invalid length or CRC.

3. AHRS message fmiA

The header is ASCII `fmiA`, with a 32-byte payload and a 40-byte complete frame. Byte offsets below start at zero.

Offset	Size	Field / type
0	4	`fmiA` header
4	2	Payload length 32, `uint16`
6	4	Device time in milliseconds, `uint32`
10, 12, 14	2 each	Acceleration X, Y, Z; all `int16`
16, 18, 20	2 each	Angular velocity X, Y, Z; all `int16`
22, 24, 26	2 each	Magnetic field X, Y, Z; all `int16`
28, 30, 32	2 each	Roll, Pitch, Yaw; all `int16`
34	4	`status`, `uint32`
38	2	CRC16, `uint16`

This firmware treats acceleration X as signed, like Y and Z. Conversion factors used by the tool are:

```
time_s = time_ms / 1000.0
acc_g = raw_acc * 8.0 / 32000.0
gyro_deg_s = raw_gyro * 500.0 / 32000.0
mag_uT = raw_mag * 500.0 / 32000.0
angle_deg = raw_angle * 180.0 / 32000.0
```

The raw acceleration, gyro, magnetic, and angle values are signed integers. Interpret `status` using individual bit masks rather than whole-word equality:

Mask	Meaning for this firmware	UI location
`0x00000001`	Magnetic field available	Magnetic-field indicator in 3D attitude
`0x00000002`	Magnetic heading valid	Magnetic-heading indicator in 3D attitude

```
mag_available = (status & 0x01) != 0
mag_heading_valid = (status & 0x02) != 0
```

The firmware sets `0x01` when `mag_manager.IsAvailable()` is true, and sets `0x02` when the last magnetic update was less than 60 seconds ago. Examples of the low two bits: `0x00` neither condition, `0x01` magnetic field only, `0x02` magnetic heading only, and `0x03` both conditions. Test the bits independently: if `0x02` appears without `0x01`, display the received bits rather than correcting them. The `0x02` bit does not by itself prove that magnetic calibration is complete. Other status bits are not confirmed for this firmware; the tool still retains the full raw value for export and debugging. Do not apply these two bit meanings to legacy status fields.

4. Magnetometer calibration frame fmib

The current device uses an ASCII `fmib` header, a 44-byte payload, and a 52-byte complete frame. Older logs may contain a 40-byte payload and a 48-byte complete frame. Each variant must pass its own length and CRC checks.

Offset	Size	Field / type
0	4	`fmib` header
4	2	Payload length 44 (40 in old logs)
6	4	Device time in milliseconds, `uint32`
10 through 38	Every 4 bytes	Indicators 1 through 8, each `float32`
42	4	Residual, `float32`
46	4	Reserved `uint32` in current frame; absent in old frame
50	2	CRC16 in current frame; offset 46 in old frame

3D mode checks all eight indicators; 2D mode checks indicators 1, 2, 4, and 7. Thresholds in order are `1, 1, 1, 0.05, 0.05, 0.05, 0.02, 0.02`. Each active value must be strictly below its threshold. Values received before the current calibration run cannot authorize saving.

The magnetic sample trace is drawn from fmiA magnetic X/Y/Z values. fmib carries calibration indicators and residual instead. These are different data sources.

5. System calibration text line $GPCAL

`$GPCAL` is a comma-separated text line, not a binary fmiA/fmib frame. Field numbers start at zero, including the header. The final field may have a `*` suffix. The current tool reads fields 2 through 5 and 7 through 13; other fields keep their protocol positions.

Field	Content
0	`$GPCAL`
1	Raw device time
2	Current average temperature; `-1` may indicate unfinished warm-up
3	Static-stage temperature
4	Static acceleration standard deviation
5	Static gyroscope standard deviation
6	Reserved
7	Calibration accuracy standard deviation; may be `-1` until complete
8	`STEP1` or `STEP2`; the tool also accepts `1` or `2`
9	Phase: `0` warm-up, `1` static capture, `2` STEP1 complete, `3` STEP2 capture, `4` all complete
10	Sample count
11	Operator instruction; interpreted by step below
12	Failure reason; `0` means no reported reason
13	Result: `0` failed, `1` running, `2` succeeded

For STEP1, field 11 follows this device-specific extension: `0` hold the current face still; `1` the bottom face is complete, turn to a side face; `2` both bottom and side are complete. These poses calibrate the IMU-to-case mounting angle. The tool also requires STEP1 phase `2` before allowing STEP2. In STEP2, complete six static faces as prompted: field 11 value `0` means hold still, and `1` means turn to the next face. Result `2` is required to confirm overall success.

Field 12 failure reasons: `1` result verification, `2` parameter storage, `3` excessive temperature rise, `4` calculation error, `5` abnormal data, `6/8` residual checks, `7/9` variance checks, `10` insufficient samples, `11` timeout. Inspect the original `$GPCAL` line when diagnosing a failure.

6. Legacy logs and scope

The tool can also replay old `fmim` (52 bytes), `fmig` (95 bytes), and `fmin` (100 bytes) records. They use a legacy additive checksum and a different time representation. Physical units that have not been confirmed are shown as `raw`. Do not reuse their time, units, or status semantics for fmiA. Unknown messages with valid length and CRC are skipped without producing a plot or export row.

Firmware transfer uses a separate APP/BOOT upgrade protocol and is outside this measurement-frame reference. Commands and fields may vary by model and firmware; retain raw logs and complete hardware validation before production integration.